Project Report

Title: MarketPulse Pro — Stock Trading Simulator

Submitted by: Karan Patil & Vansh Parekh
Student Roll No: 40,37
Student Enrollment ID: 2403111011,2403111014
Submitted to: Dr. R. S. Khamitkar
Department: Electronics and Computer Engineering
Date:

1. Project Overview

MarketPulse Pro is a Python-based stock trading dashboard that simulates a real-time market environment with a clean and modern dark-themed interface inspired by platforms like Groww.

The application generates live price movements for a selected set of major Indian stocks—RELIANCE, TCS, INFY, HDFCBANK, and WIPRO—using a mathematical simulation model based on drift and volatility. This creates realistic market behaviour without relying on external APIs.

Users can easily buy and sell stocks through an interactive interface, monitor their portfolio value, and track profit or loss in real time. The system also maintains a detailed transaction history, helping users understand their trading performance.

Overall, MarketPulse Pro offers a smooth and practical way to explore stock market concepts, combining simulation, visualization, and user interaction in a single application.

2. Problem Statement

Aspiring traders and students who are learning about financial markets often do not have access to a safe and risk-free environment to practice trading. Real trading platforms involve actual financial risk, while most paper-trading tools are either web-based, require API access, or depend on a stable internet connection. This creates a barrier for beginners who want to understand market behaviour and trading strategies in a simple and controlled setting.

This project addresses these challenges by developing a fully offline, self-contained stock trading simulator that provides both functionality and a realistic user experience.

The key objectives of the system are:

- To generate realistic stock price movements using mathematical simulation techniques without relying on external APIs.
- To provide a clean, professional, and user-friendly interface inspired by modern fintech applications.
- To allow users to practice buying and selling stocks, helping them understand portfolio management and profit/loss calculations.
- To demonstrate important programming concepts such as Object-Oriented Programming (OOP), threading for real-time updates, Tkinter for GUI development, and Matplotlib for data visualization.

Overall, the project aims to create an accessible learning tool that combines financial concepts with practical programming implementation.

3. Technology Stack

Components	Technology / Library
Programming Language	*Python 3.9+*
GUI Framework	*Tkinter (standard library)*
Charting	*Matplotlib 3.7+ (TkAgg backend)*
Price Simulation	*Python math, random (GBM model)*
Concurrency	*Python threading (daemon thread)*
IDE	*VS Code / Python IDLE*

4. Project Structure

The project follows a clean, modular MVC-inspired architecture. Each concern is isolated in its own file:

```
├── main.py          ← The blazing fast entry point (only 5 lines!)
├── config.py        ← All constants: tickers, aesthetic tokens, UI colors, fonts
├── engine.py        ← Heavy lifting: Price simulation (GBM), StockData, PriceEngine
├── portfolio.py     ← Order logic, Position Management, P&L generation, TX history
├── ui.py            ← Tkinter GUI Views, Canvas Widgets, Component Factories
├── requirements.txt ← Dependencies
└── .gitignore       ← Excluded files
```

4. Implementation

The implementation of MarketPulse Pro is organized into four main functional layers, each responsible for a specific part of the system:

---

1. Price Simulation (engine.py)
This layer is responsible for generating realistic stock price movements. It uses the **Geometric Brownian Motion (GBM)** model, where the next price depends on the current price along with **drift (trend)** and **volatility (random fluctuation)**. A **daemon thread** continuously updates stock prices at regular intervals (around 1.2 seconds) and notifies the UI so that changes are reflected in real time.

---

2. Portfolio Management (portfolio.py)
This module handles all trading operations. The **buy()** function deducts cash and updates the **average buying price**, while the **sell()** function adds cash and reduces the number of shares held. It also calculates important metrics like **unrealized profit/loss (P&L)** and **net worth**, allowing users to track their performance based on current market prices.

---

3. UI Rendering (ui.py)
The user interface is built entirely using **Tkinter** with a clean, **dark-themed layout inspired by Groww**. The design follows a **three-column structure**:

- Left: Stock watchlist
- Center: Live chart using **Matplotlib** (price trend and momentum)
- Right: Buy/Sell order panel

A separate **Portfolio section** displays key information such as holdings, profit/loss, and transaction history. All updates are handled safely using **self.after()**, ensuring smooth and crash-free UI performance.

---

4. Configuration (config.py)
This file acts as a **central configuration hub**. It stores all important constants such as stock details, drift and volatility values, and UI settings. Having a single source of configuration improves **maintainability and scalability** of the project.

---

6. Code Snippet

main.py — Entry Point (5 lines):

```
from app.ui import App


if __name__ == "__main__":
    app = App()
    app.mainloop()
```

engine.py — GBM Price Step:

```
def _gbm_step(self) -> float:
    shock  = random.gauss(0, 1)
    factor = math.exp(self.drift + self.volatility * shock)
    return self.price * factor
```

portfolio.py — Buy Logic:

```
def buy(self, symbol, name, qty, price):
    cost = qty * price
    if cost > self.cash:
        return False, "Insufficient funds."
    self.cash -= cost
    # Update volume-weighted average price
    if symbol in self.holdings:
        h = self.holdings[symbol]
        total_qty   = h.qty + qty
        h.avg_price = (h.avg_price * h.qty + price * qty) / total_qty
        h.qty       = total_qty
    else:
        self.holdings[symbol] = Holding(symbol, name, qty, price)
```

```


    return True, f"Bought {qty} x {symbol} @ Rs.{price:,.2f}"
```

5. Sample Test Cases

Action	Input	Expected Output
Buy 10 RELIANCE @ 2850	qty=10, symbol=RELIANCE	Cash reduced by Rs.28,500; holding created
Sell 5 RELIANCE	qty=5, symbol=RELIANCE	Cash credited; quantity updated to 5
Sell more than held	qty=20, holding=5	Error: Only 5 shares available
Buy with insufficient funds	cost > available cash	Error: Insufficient funds
View Portfolio	Portfolio tab click	Net worth, P&L, positions displayed
Price update	Engine tick	Sidebar, chart, header refresh every 1.2s

6. Challenges Faced

Thread Safety: Matplotlib and Tkinter both require updates on the main thread. Solved using self.after(0, callback) to safely schedule all UI refreshes from the engine's background thread.

OHLC Candle Aggregation: Accumulating Open/High/Low/Close across multiple ticks required careful state tracking inside StockData, flushing a completed candle every CANDLE_PERIOD ticks.

Realistic Price Simulation: Pure random walk produces unrealistic drift. GBM with per-stock drift and volatility parameters produces more natural, slightly-trending price curves.

Modular UI Organization: Keeping ui.py manageable without splitting into more files required careful grouping of helper methods and factory functions (lbl, btn, sep).

7. Results and Observations

Figure 1: MarketPulse Pro – Live Stock Dashboard with Candlestick Chart and Trading Panel

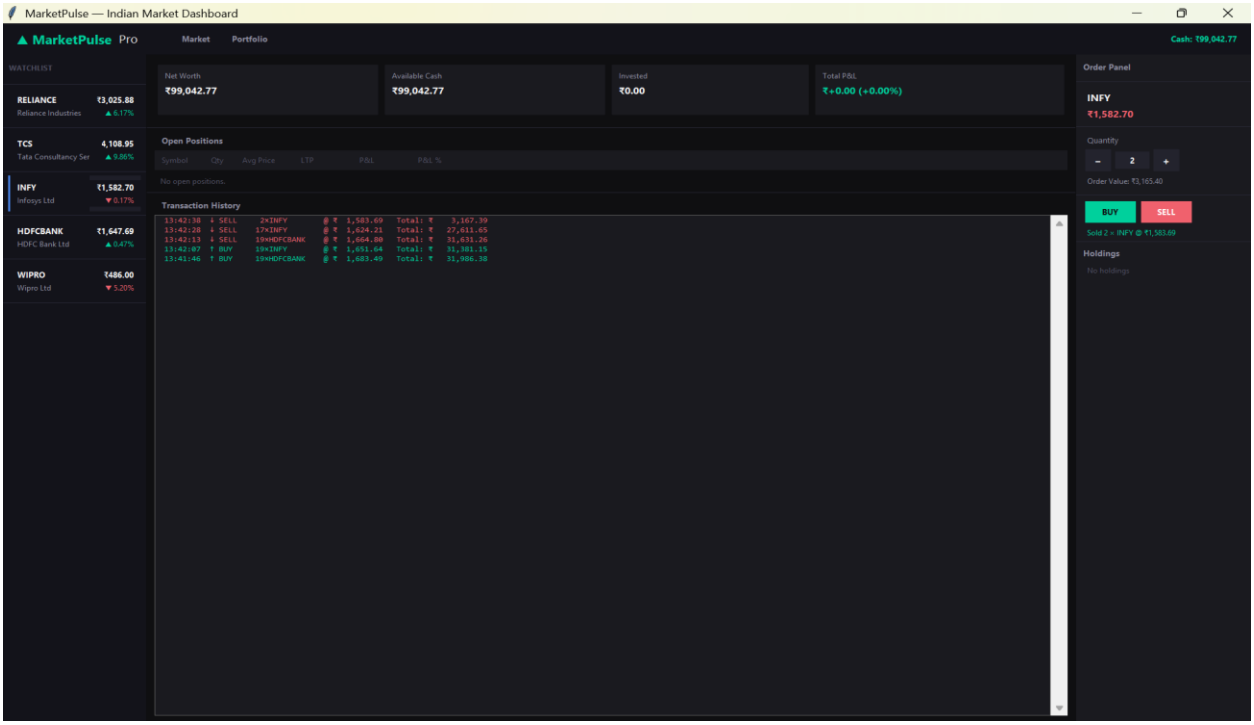

Figure 2: MarketPulse Pro – Portfolio Dashboard Showing Net Worth, Holdings, and Transaction History

The above result and observation shows:-

- Application runs smoothly with all stocks updating at ~1.2-second intervals without UI lag.
- Candlestick charts correctly show bullish (green) and bearish (red) trends.
- Portfolio updates in real time with accurate P&L calculations.
- Buy/Sell validations (insufficient funds, insufficient holdings) work correctly.
- Modular design allows easy addition of new stocks via config.py.

Project Submission

Repository Link: https://github.com/Karanpatil1201/MarketPulse

8. Signatures

Student Signature: _____

Guide Signature: _____

Date: _____